

Data Handling and Problem Solving

Instructions:

1. All questions are **mandatory**. No question should be skipped.
 2. **Do not use AI tools or external help**. You must be ready to **explain your logic and code** during evaluation.
-

Question 1: (10 Marks)

Given an integer array `nums`, move all `0`'s to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

```
Input: nums = [0,1,0,3,12]
Output: [1,3,12,0,0]
```

Example 2:

```
Input: nums = [0]
Output: [0]
```

Constraints:

- `1 <= nums.length <= 104`
 - `231 <= nums[i] <= 231 - 1`
-

Question 2: (5 Marks)

Given an array `arr` and a chunk size `size`, return a **chunked** array.

A **chunked** array contains the original elements in `arr`, but consists of subarrays each of length `size`. The length of the last subarray may be less than `size` if `arr.length` is not evenly divisible by `size`.

Please solve it without using lodash's `_.chunk` function.

Example 1:

Input: `arr = [1,2,3,4,5]`, `size = 1`

Output: `[[1],[2],[3],[4],[5]]`

Explanation: The `arr` has been split into subarrays each with 1 element.

Example 2:

Input: `arr = [1,9,6,3,2]`, `size = 3`

Output: `[[1,9,6],[3,2]]`

Explanation: The `arr` has been split into subarrays with 3 elements. However, only two elements are left for the 2nd subarray.

Example 3:

Input: `arr = [8,5,3,2,6]`, `size = 6`

Output: `[[8,5,3,2,6]]`

Explanation: Size is greater than `arr.length` thus all elements are in the first subarray.

Example 4:

Input: `arr = []`, `size = 1`

Output: `[]`

Explanation: There are no elements to be chunked so an empty array is returned.

Constraints:

- `arr` is a string representing the array.
- `2 <= arr.length <= 105`
- `1 <= size <= arr.length + 1`

Question no 3: (5 Marks)

Given two arrays `arr1` and `arr2`, return a new array `joinedArray`. All the objects in each of the two inputs arrays will contain an `id` field that has an integer value.

`joinedArray` is an array formed by merging `arr1` and `arr2` based on their `id` key. The length of `joinedArray` should be the length of unique values of `id`. The returned array should be sorted in **ascending** order based on the `id` key.

If a given `id` exists in one array but not the other, the single object with that `id` should be included in the result array without modification.

If two objects share an `id`, their properties should be merged into a single object:

- If a key only exists in one object, that single key-value pair should be included in the object.
- If a key is included in both objects, the value in the object from `arr2` should override the value from `arr1`.

Example 1:

Input:

```
arr1 = [  
  {"id": 1, "x": 1},  
  {"id": 2, "x": 9}  
],  
arr2 = [  
  {"id": 3, "x": 5}  
]
```

Output:

```
[  
  {"id": 1, "x": 1},  
  {"id": 2, "x": 9},  
  {"id": 3, "x": 5}  
]
```

Explanation: There are no duplicate ids so `arr1` is simply concatenated with `arr2`.

Example 2:

Input:

```
arr1 = [  
  {"id": 1, "x": 2, "y": 3},  
  {"id": 2, "x": 3, "y": 6}
```

```

],
arr2 = [
  {"id": 2, "x": 10, "y": 20},
  {"id": 3, "x": 0, "y": 0}
]

```

Output:

```

[
  {"id": 1, "x": 2, "y": 3},
  {"id": 2, "x": 10, "y": 20},
  {"id": 3, "x": 0, "y": 0}
]

```

Explanation: The two objects with id=1 and id=3 are included in the result array without modification. The two objects with id=2 are merged together. The keys from arr2 override the values in arr1.

Example 3:

```

Input:
arr1 = [
  {"id": 1, "b": {"b": 94}, "v": [4, 3], "y": 48}
]
arr2 = [
  {"id": 1, "b": {"c": 84}, "v": [1, 3]}
]

```

```

Output: [
  {"id": 1, "b": {"c": 84}, "v": [1, 3], "y": 48}
]

```

Explanation: The two objects with id=1 are merged together. For the keys "b" and "v" the values from arr2 are used. Since the key "y" only exists in arr1, that value is taken from arr1.

Constraints:

- `arr1` and `arr2` are valid JSON arrays
- Each object in `arr1` and `arr2` has a unique integer `id` key
- `2 <= JSON.stringify(arr1).length <= 106`
- `2 <= JSON.stringify(arr2).length <= 106`

